REQUIRED EDITIONS

Available in: Lightning Experience
Available in: **Enterprise** and **Unlimited** Editions with Life Sciences Cloud, Life Sciences Cloud for Customer Engagement Add-on license, and the Life Sciences Customer Engagement managed package.

Step 1: Outline the Workflow

Identify the business process that you want to automate. Then, break down the workflow into logical steps or stages. For example, a common Inquiry process includes stages such as New, Working, Escalated, and Closed.

Step 2: Define Workflow Objects and Permissions

Identify the parent object that you want the workflow to run on, as well as any related objects. Determine the permissions users need to create, edit, or delete parent or child records at each stage in the object's workflow.

Step 3: Identify Fields for Workflow Conditions

Each workflow stage operation includes conditions that must be met so that users can work with records and take actions. Identify the fields on the parent object or related objects that are required to set up these conditions.

Step 4: Plan Actions for Each Workflow Stage

Workflow actions can update records, launch a platform event, invoke a Lightning web component, or open a custom URL. Outline the actions that you want to make available to users at each stage in the workflow. This way, you can identify the record fields or other components that the actions use or update.

Step 5: Consider Validation Rules and Checklists

Validation rules and checklists help you maintain data integrity and enforce business rules. For validation rules, consider what conditions must be met for a record to move to the next stage in the workflow. For checklists, identify any necessary steps that you want to outline for users so that they know how to move a record to the next stage.

Manage Life Sciences Workflows

To streamline your business processes, create tailored Life Sciences Cloud for Customer Engagement

workflows.

REQUIRED EDITIONS

Available in: Lightning Experience
Available in: **Enterprise** and **Unlimited** Editions with Life Sciences Cloud, Life Sciences Cloud for Customer Engagement Add-on license, and the Life Sciences Customer Engagement managed package.

You can manage Life Sciences workflows from the Workflow Configuration tile in the Admin Console. From the Workflow Paths page, you can:

- Search the list of workflow paths, and sort by column to find the workflow you need.
- Get details about each workflow path, such as the object and record type it applies to, its status, and its controlling field.
- Create a workflow path.
- Clone a workflow path.
- Open or edit a workflow path to modify its details, activate it, or deactivate it.

After you configure a Life Sciences workflow, you can use Data Loader to move the workflow configuration to another org.

Configure Life Sciences Workflow Actions
Before you configure a workflow path, create the actions that users can perform on an object. When you define the workflow path, you can assign these actions at each stage in the workflow.

Configure Life Sciences Workflow Paths
Create workflow paths for each object and record type to meet your business needs. At each stage of the record's lifecycle, define user permissions and the actions that users can take.

Configure Custom Scripts for Life Sciences Workflows
To make sure that records meet your business rules before the Life Sciences Customer Engagement workflow continues, create and manage custom scripts. Each time a user performs an action, validation scripts run to prevent incorrect actions or alert users about errors. Checklist scripts show an info icon on Update Record actions so that users can see the next steps to take.

Assign Custom Scripts to Life Sciences Workflow Objects
To enforce business rules throughout the workflow, update the validation scripts and checklists that apply to each stage object.

See Also
Data Loader Guide

Configure Life Sciences Workflow Actions

Before you configure a workflow path, create the actions that users can perform on an object. When you define the workflow path, you can assign these actions at each stage in the workflow.

REQUIRED EDITIONS

Available in: Lightning Experience
Available in: **Enterprise** and **Unlimited** Editions with Life Sciences Cloud, Life Sciences Cloud for Customer Engagement Add-on license, and the Life Sciences Customer Engagement managed package.

USER PERMISSIONS NEEDED	
To create workflow actions:	Life Sciences Commercial Admin permission set

You can create four types of workflow actions.

- Update Record actions edit records in some way, for example, by moving the record to a new status.
- Platform Event actions trigger real-time processes. For example, a platform event can launch a flow.
- Component actions invoke a Lightning web component. For example, you can open a component that shows a list of available document templates.
- Custom Actions support custom functionality, such as opening an external URL. You create custom actions from the Quick and Custom Action Administration page in the Admin Console. Actions must have the Stage Path entity type and the URL action type. See Create Custom Actions.

1. From the App Launcher, find and select **Admin Console**.
2. Select **Workflow Configuration**, and then select **Workflow Actions**.
   To see and search for available actions, select the tabs for each action type.
3. To create an action, click **New**, and select the action type that you want to create.
4. Enter the action's basic information, such as its name, the button label, and the object on which it's available.
5. For an Update Record action, enter these details.
   a. Select the field on this object that the action updates, and enter the field's default value.
   b. In the Target Object section, select an additional object, add the fields to update, and enter the default field values.
      When this action is triggered, a window opens where users can review, update, and save changes to these fields.
   c. In the Platform Event Parameters section, select a platform event to publish as part of this action, add the fields to include, and specify the default values.
      The platform event details are sent as parameters to workflows, Apex processes, or other asynchronous processes that are subscribed to this platform event.
6. For a Platform Event action, enter these details.
   a. In the Target Object section, select an additional object, add the fields to update, and enter the default field values.
      When this action is triggered, a window opens where users can review, update, and save changes to these fields.
   b. In the Platform Event Parameters section, select a platform event to publish as part of this action, add the fields to include, and specify the default values.
      The platform event details are sent as parameters to workflows, Apex processes, or other

asynchronous processes that are subscribed to this platform event.

7. For a Component action, enter these details.
   a. In the Component Parameters section, enter the name of the Lightning web component to open from this action.
   b. To populate field values when the component loads, enter component parameters.
      For example, for an action that opens a component to capture a signature, add these parameters to set these values when the component loads:
      `"status":"Signed","disclaimerText":"Disclaimer Text"`.
8. For a Custom Action, select a custom URL action to open an external link.
9. Save your changes.

After you create actions, you can see action details and parameters on the Workflow Actions page. You can also edit, clone, or delete actions.

See Also

Define and Manage Platform Events
Lightning Web Components Developer Guide: Create Lightning Web Components

Configure Life Sciences Workflow Paths

Create workflow paths for each object and record type to meet your business needs. At each stage of the record's lifecycle, define user permissions and the actions that users can take.

REQUIRED EDITIONS

Available in: Lightning Experience
Available in: **Enterprise** and **Unlimited** Editions with Life Sciences Cloud, Life Sciences Cloud for Customer Engagement Add-on license, and the Life Sciences Customer Engagement managed package.

USER PERMISSIONS NEEDED	
To configure workflow paths:	Life Sciences Commercial Admin permission set

1. From the App Launcher, find and select **Admin Console**.
2. Select **Workflow Configuration**, and then select **Workflow Paths**.
3. Click **New**, and give the workflow path a name.
4. Select the object and record type to create the workflow path for.
5. To determine the stages in the workflow path, select the controlling picklist field.
   The workflow builder shows stages for each picklist value that's assigned to the controlling field for that object and record type.
6. To open the workflow builder and define the workflow path, click **Continue**.
   The workflow builder opens on the first stage of the workflow. To select a different stage, click its name in the progress bar at the top of the page.
7. Define a stage operation, which groups the conditions that control the permissions users have and the

actions they can take on records in this stage.

a. Under Stage Operations, click the plus icon.
b. Name the operation.
c. Select whether users can create, edit, or delete records at this stage when conditions are met. Standard Salesforce object and field-level permissions still apply.
d. Enter a priority for this operation.
   When multiple stage operations apply to a record, the highest priority operation takes precedence, and users are granted those permissions.
e. Save your changes.

8. Define stage operation conditions that must be met to grant users permissions and show configured actions for this stage and operation.
   a. Select **Stage Operation Conditions**, and then click the plus icon.
   b. Select the field to evaluate.
      You can select fields on this object or a related object.
   c. Set the operator for the condition, and enter the required field value.
   d. Add more conditions as needed.
      All conditions must be met to grant permissions and actions.
9. Define the stage operation actions, which determine the actions that users can perform when conditions are met for this stage.
   a. Select **Stage Operation Actions**.
   b. Click **Add Actions**, and then select the type of action to add.
   c. Select the actions to add to the page, and then save your changes.
   d. Add more actions as needed. To reorder the actions, use the arrows.
10. In the Other Permissions tab, grant users access to Salesforce Files so that users can manage documents and attachments at this stage when conditions are met.
    - To prevent users from working with documents and attachments, select **Salesforce Files** but don't select any permissions.
    - To enable users to work with documents and attachments, select **Salesforce Files**, and then select whether users can create, edit, or delete files.
11. Add or edit other stage operations for this stage, if needed.
    a. To add another parent stage operation, click the plus icon next to Stage Operations.
    b. To add a child stage operation, click the plus icon next to a stage's name.
       Child stage operations control access to child objects of the workflow path's object.
       For example, in an Inquiry workflow path, create a child stage operation to control when users can create, edit, or delete Inquiry Question records.
    c. To edit a stage operation, select it under Stage Operations, and then click the pencil icon.
    d. To delete a stage operation, select it under Stage Operations, and then click the trash can icon.
    e. To clone a stage operation and make minor changes, select it under Stage Operations, and then click the copy icon.
12. Similarly, configure at least one stage operation for each stage in the workflow.
    When a stage is configured, a green checkmark appears in the progress bar.
13. After your workflow is complete, activate it to make it available to that object or object type, or save your changes and activate it later.
14. To create a copy of this workflow path, click **Clone**.

See Also

Considerations for Life Sciences Workflows

Configure Custom Scripts for Life Sciences Workflows

To make sure that records meet your business rules before the Life Sciences Customer Engagement workflow continues, create and manage custom scripts. Each time a user performs an action, validation scripts run to prevent incorrect actions or alert users about errors. Checklist scripts show an info icon on Update Record actions so that users can see the next steps to take.

REQUIRED EDITIONS

Available in: Lightning Experience
Available in: **Enterprise** and **Unlimited** Editions with Life Sciences Cloud, Life Sciences Cloud for Customer Engagement Add-on license, and the Life Sciences Customer Engagement managed package.

USER PERMISSIONS NEEDED	
To configure custom scripts:	Life Sciences Commercial Admin permission set

First, configure custom scripts for the validations that you want to perform or for the checklists that you want to apply. See Create Lightning Web Components and Custom Scripts for Life Sciences.

Then, you can apply the custom scripts from the Admin Console in Life Sciences Customer Engagement.

1. From the App Launcher, find and select **Admin Console**.
2. Select **Workflow Configuration**, and then select **Custom Scripts**.
3. Click **New**, and give the script a name.
4. Enter the name of the Lightning web component that you created.
5. Select whether this is a script for a checklist or for validation.
6. Save your changes.
7. To sync changes to the customer script after you update the Lightning web component, select **Refresh**.

After you configure custom scripts, assign them to workflow objects.

Assign Custom Scripts to Life Sciences Workflow Objects

To enforce business rules throughout the workflow, update the validation scripts and checklists that apply to each stage object.

REQUIRED EDITIONS

Available in: Lightning Experience

Available in: **Enterprise** and **Unlimited** Editions with Life Sciences Cloud, Life Sciences Cloud for Customer Engagement Add-on license, and the Life Sciences Customer Engagement managed package.

USER PERMISSIONS NEEDED	
To assign custom scripts to workflow objects:	Life Sciences Commercial Admin permission set

Stage object records are created automatically after you create and save a Life Sciences Cloud for Customer Engagement workflow path. One stage object record is created for each object that has a workflow. Validations and checklists apply to all object types.

1. From the App Launcher, find and select **Admin Console**.
2. Select **Workflow Configuration**, and then select **Stage Objects**.
3. Find the object to apply custom scripts to, and then select **Edit** from the row-level actions.
4. In the Validation Script field, select the custom script that you created for validation.
5. In the Checklist Script field, select the custom script that you created for the checklist items.
6. Save your changes.
   After you save your changes, custom scripts run automatically as long as the object has an active workflow.

See Also

Configure Custom Scripts for Life Sciences Workflows
Custom Scripts for Life Sciences

Considerations for Life Sciences Workflows

When you configure a Life Sciences workflow, keep these considerations in mind.

REQUIRED EDITIONS

Available in: Lightning Experience
Available in: **Enterprise** and **Unlimited** Editions with Life Sciences Cloud, Life Sciences Cloud for Customer Engagement Add-on license, and the Life Sciences Customer Engagement managed package.

Conflicting Workflow Paths for Parent and Child Objects

When two separate workflow paths control the record's permissions, the most restrictive permissions apply. Two workflow paths can control permissions to an object record at the same time when:

- A parent object's workflow path applies a child stage operation for the child object.
- The child object has its own workflow path and stage operation.

For example, you can have separate workflow paths for the Inquiry object and the child Inquiry Question

object. The Inquiry Question workflow path includes a stage operation that grants users only the Delete permission when the response status is Responded. The Inquiry workflow path includes a child stage operation that grants users the Create, Edit, and Delete permissions on Inquiry Question records when the response status is Responded or Not Responded. Because the Inquiry Question workflow path is more restrictive, users can't create or edit Inquiry Question records with a Responded status.

Workflow Paths for Objects with Record Types

For objects with record types:

- You must configure a workflow path for every record type. If you create workflow paths only for some record types, users can't create, edit, or delete records of other types.
- To determine the stages in the workflow path, select the picklist values to use for the workflow's controlling field on each record type.

Workflow Path Stages

Each workflow stage requires at least one stage operation. If a stage doesn't have a stage operation:

- Users can't create, update, or delete records when the record is at that stage.
- The stage doesn't appear in the progress bar at the top of the record detail page.

Workflow Path Field Lengths

Text fields in the workflow path support a maximum length of up to 255 characters. With this length limit, you can reference parent objects in the workflow path, for example, in stage operation conditions.

Stage Operation Priority

When a workflow stage has multiple operations, a record in that stage can meet the conditions for more than one operation. To make sure that the workflow applies the correct operation and grants users the correct permissions, assign each operation a priority.

For example, the New stage in the Inquiry workflow path has two operations:

- Inquiry New Stage Operation 1" has condition "City==San Francisco AND State==CA"
- Inquiry New Stage Operation 2" has condition "State==CA"

A new inquiry record in San Francisco, California, meets both conditions. Assign the higher priority to Operation 1 to grant users those permissions.

Workflow Record Update Actions

When a user runs a Record Update action, the action runs in this order.

1. The validation script runs.
2. The window opens where users can review, update, and save changes to other object fields. For example, a window opens where a user can enter additional information about why they're canceling an inquiry.
3. The primary record is updated.
4. The platform event fires.

Custom Scripts for Life Sciences

Custom scripts are programmatic tools for data validation across Life Sciences Cloud for Customer Engagement on desktop and both online and offline in the Life Sciences Cloud mobile app. Custom scripts are used in Life Sciences workflow management and visit management.

REQUIRED EDITIONS

Available in: Lightning Experience
Available in: **Enterprise** and **Unlimited** Editions with Life Sciences Cloud, Life Sciences Cloud for Customer Engagement Add-on license, and the Life Sciences Customer Engagement managed package.

Custom scripts execute during Life Sciences workflow actions to support more complex business scenarios than standard Salesforce validation rules. Unlike Salesforce validation rules, custom scripts consolidate validation logic and can query any accessible database data, rather than just related records. Custom scripts also offer more dynamic user feedback, providing warnings and custom error messages when users execute actions.

You can create these types of custom scripts.

- Checklist: Checklist custom scripts are used in Life Sciences Cloud for Customer Engagement workflows. Checklist scripts help users understand all of the steps to take before they can move to the next stage in the workflow. These scripts are executed when a user clicks the info icon on Record Update actions.
- Validation: Validation custom scripts are used in Life Sciences Cloud for Customer Engagement workflows. Validation scripts make sure that business rules are met before users can move a record to the next stage in a workflow. These scripts are executed when a user runs any workflow action.
- Visit Action Validation: Visit action validation custom scripts validate business rules before a user can sign and submit a visit. These scripts are executed when a user runs an action to sign or submit visits.

Understand the Format and Output for Life Sciences Custom Scripts

Understand the format and output for custom scripts in Life Sciences Cloud for Customer Engagement.

Best Practices for Life Sciences Custom Scripts

Follow these best practices when you create custom scripts for Life Sciences Cloud for Customer Engagement.

Test Custom Scripts for Life Sciences
Because custom scripts aren't typical Lightning web components, you can't test them in the same way. Write custom Jest tests to test the Lightning web components that you create as custom scripts for workflows in Life Sciences Cloud for Customer Engagement.

Troubleshoot Custom Scripts for Life Sciences
To troubleshoot and debug custom scripts for Life Sciences Cloud for Customer Engagement workflow validation, use Chrome Developer Tools. The Console tab shows log information, while the Network tab helps monitor network requests, responses, and performance.

Example Visit Action Validation Script
Visit Action Validation Custom Scripts help make sure that users follow business rules before signing or submitting a visit. These scripts run when the user selects the Sign or Submit action.

Life Sciences Custom Scripts Reference
Understand the available JavaScript classes, functions, and variables that you can include in a Lightning web component for custom scripts in Life Sciences Cloud for Customer Engagement.

Understand the Format and Output for Life Sciences Custom Scripts

Understand the format and output for custom scripts in Life Sciences Cloud for Customer Engagement.

REQUIRED EDITIONS

Available in: Lightning Experience
Available in: **Enterprise** and **Unlimited** Editions with Life Sciences Cloud, Life Sciences Cloud for Customer Engagement Add-on license, and the Life Sciences Customer Engagement managed package.

Format

You create custom scripts as headless Lightning web components, meaning that the component includes only the JavaScript file and the metadata configuration file. See Create Lightning Web Components.

In the LWC, contain custom script logic in a self-calling function.

```
(() => {
    // Add custom script logic here.
    ...
})();
```

Output

Custom scripts return an array of JavaScript objects with two defined properties.

Property	Type	Details
title	String	The message that users see. This can be a custom label name that can be translated to match the current user's language.
status	String	The type of message. Statuses are: • success: Shows a success message for checklist scripts. Not displayed for validation scripts. • warning: Shows a warning message for checklist scripts. For validation scripts, if there are warnings but not errors, users see all warnings in one window, and they can choose to continue. • error: Shows an error message for checklist scripts. Shows an error window for validation scripts.

See this example output.

```
return [
    {
        title: "Success Message",
        status: "success",
    },
    {
        title: "Success_Message_Custom_Label",
        status: "success",
    },
    {
        title: "Warning Message",
        status: "warning",
    },
    {
        title: "Warning_Message_Custom_Label",
        status: "success",
```

```
    },
    {
        title: "Error Message",
        status: "error",
    },
    {
        title: "Error_Message_Custom_Label",
        status: "success",
    }
];
```

Best Practices for Life Sciences Custom Scripts

Follow these best practices when you create custom scripts for Life Sciences Cloud for Customer Engagement.

REQUIRED EDITIONS

Available in: Lightning Experience
Available in: **Enterprise** and **Unlimited** Editions with Life Sciences Cloud, Life Sciences Cloud for Customer Engagement Add-on license, and the Life Sciences Customer Engagement managed package.

Asynchronous Calls

Database queries are synchronous on the Life Sciences Cloud mobile app but asynchronous on desktop. To support asynchronous calls, enclose each validation in an `async` function.

```
(() => {
    async function inquiryQuestionsValidation() {
        try {
            let inquiryQuestions = await db.query(
                "InquiryQuestion",
                await new ConditionBuilder(
                    "InquiryQuestion",
                    new FieldCondition("InquiryId", "=", getRecordId(record))
                ).build(),
                ["Id", "Name"]
            );

            if (
                inquiryQuestions === null ||
```

```
                inquiryQuestions === undefined ||
                inquiryQuestions.length === 0
            ) {
                return {
                    title: "No Inquiry Questions Found",
                    status: "error",
                };
            }

            return {
                title: "Inquiry Questions Added",
                status: "success",
            }

        } catch(error) {
            return {
                title: "Caught Exception During Inquiry Questions Validation",
                status: "error"
            }
        }
    }

    function getRecordId(record) {
        let recordId = record.stringValue("Id");
        return recordId ? recordId : record.stringValue("uid");
    }

    return [inquiryQuestionsValidation()];
})();
```

Database Queries

- To improve performance, always specify fields in database queries.
- Rather than making multiple WHERE clause queries to the database, filter queries in the JavaScript instead.

Error Handling

To make sure that you “catch” JavaScript errors, use a `Try-Catch` block in all validation functions in custom scripts. Otherwise, when JavaScript errors occur, validation rules don’t show in the UI.

If `enableAccessErrors()` is called before executing a function to retrieve field values, the function returns a JavaScript error for the inaccessible field. If an error occurs but isn’t caught by a `Try-Catch`

block:

- On desktop, users see only a field access error for checklist and validation scripts.
- On the Life Sciences Cloud mobile app, users see a field access error for checklist scripts, but validation scripts fail.

Testing

Because errors can sometimes occur in only one environment, we recommend testing custom scripts on both desktop and in the Life Sciences Cloud mobile app.

Test Custom Scripts for Life Sciences

Because custom scripts aren't typical Lightning web components, you can't test them in the same way. Write custom Jest tests to test the Lightning web components that you create as custom scripts for workflows in Life Sciences Cloud for Customer Engagement.

REQUIRED EDITIONS

Available in: Lightning Experience
Available in: **Enterprise** and **Unlimited** Editions with Life Sciences Cloud, Life Sciences Cloud for Customer Engagement Add-on license, and the Life Sciences Customer Engagement managed package.

See Test Lightning Web Components.

1. Load the script.
   a. Read the script's contents from the file system.
   b. Create a new function that accepts all necessary parameters for the script and includes the script's content as the final parameter.
   The parameters that you pass into the function should be mocked versions of the classes and variables that are used in the script. The last parameter should be `return`, followed by the content of your script file.
   ```
   // Load the script
   const scriptContent = fs.readFileSync(path.resolve(__dirname, '../inquiryQ
   uestionsValidation.js'));
   const scriptFunction = new Function('env', 'record', 'db', 'ConditionBuild
   er', 'FieldCondition', `return ${scriptContent.toString()}`);
   ```
2. Mock custom script classes and variables.
   To create custom script tests, you must create mocked versions of any out-of-the-box classes and variables and pass them in during testing. If you don't pass in mocked classes and variables, test failures occur.

```
// Mock the classes and variables that are used in the script
const mockEnv = {
    getOption: jest.fn(),
};
const mockDb = {
    query: jest.fn(),
};
const mockRecord = {
    stringValue: jest.fn(),
};
const mockConditionBuilder = jest.fn().mockImplementation(() => ({
    build: jest.fn().mockResolvedValue({}),
}));
const mockFieldCondition = jest.fn();
```

3. Set up a test using this structure.
   a. Mock the data that's necessary for your test scenario.
   b. Invoke your script's function, and provide mocked classes and variables.
   c. To make sure that the complete result is available, allow all validation functions to resolve.
   d. Verify the results.

```
// Mock the db.query method to return the desired value
mockDb.query.mockResolvedValue([{ Id: '1', Name: 'Question 1' }]);

// The scriptFunction returns an array containing a Promise
const promiseArray = scriptFunction(mockEnv, mockRecord, mockDb, mockConditi
onBuilder, mockFieldCondition);

// Wait for the Promise in the array to resolve
const result = await Promise.all(promiseArray);

// Assert the result is as expected
expect(result).toEqual([{
    title: 'Inquiry Questions Added',
    status: 'success',
}]);
```

Example Test Suite for Life Sciences Custom Scripts
This example provides a test suite for a custom script in Life Sciences Cloud for Customer Engagement.

Example Test Suite for Life Sciences Custom Scripts

This example provides a test suite for a custom script in Life Sciences Cloud for Customer Engagement.

This example Jest test suite is designed to test the example custom script in Best Practices for Life Sciences Custom Scripts.

```
import fs from 'fs';
import path from 'path';

// Load the script
const scriptContent = fs.readFileSync(path.resolve(__dirname, '../inquiryQuest
ionsValidation.js'));
const scriptFunction = new Function('env', 'record', 'db', 'ConditionBuilder',
'FieldCondition', `return ${scriptContent.toString()}`);

// Mock the classes and variables that are used in the script
const mockEnv = {
    getOption: jest.fn(),
};
const mockDb = {
    query: jest.fn(),
};
const mockRecord = {
    stringValue: jest.fn(),
};
const mockConditionBuilder = jest.fn().mockImplementation(() => ({
    build: jest.fn().mockResolvedValue({}),
}));
const mockFieldCondition = jest.fn();


describe('inquiryQuestionsValidation', () => {

    beforeEach(() => {
        // Clear all mocks and reset modules to ensure a clean state
        jest.clearAllMocks();
    });

    test('should return Inquiry Questions Added when there are inquiry questio
ns', async () => {
        // Mock the db.query method to return the desired value
        mockDb.query.mockResolvedValue([{ Id: '1', Name: 'Question 1' }]);

        // The scriptFunction returns an array containing a Promise
        const promiseArray = scriptFunction(mockEnv, mockRecord, mockDb, mockC
onditionBuilder, mockFieldCondition);
```

```
        // Wait for the Promise in the array to resolve
        const result = await Promise.all(promiseArray);

        // Assert that the result is as expected
        expect(result).toEqual([{
            title: 'Inquiry Questions Added',
            status: 'success',
        }]);
    });

    test('should return No Inquiry Questions Found when there are no inquiry q
uestions', async () => {
        // Mock the db.query method to return the desired value
        mockDb.query.mockResolvedValue([]);

        // The scriptFunction returns an array containing a Promise
        const promiseArray = scriptFunction(mockEnv, mockRecord, mockDb, mockC
onditionBuilder, mockFieldCondition);

        // Wait for the Promise in the array to resolve
        const result = await Promise.all(promiseArray);

        // Assert that the result is as expected
        expect(result).toEqual([{
            title: 'No Inquiry Questions Found',
            status: 'error',
        }]);
    });

    test('should return Caught Exception During Inquiry Questions Validation w
hen there is an error', async () => {
        // Mock the db.query method to return the desired value
        mockDb.query.mockRejectedValue(new Error('Error'));

        // The scriptFunction returns an array containing a Promise
        const promiseArray = scriptFunction(mockEnv, mockRecord, mockDb, mockC
onditionBuilder, mockFieldCondition);

        // Wait for the Promise in the array to resolve
        const result = await Promise.all(promiseArray);

        // Assert that the result is as expected
        expect(result).toEqual([{
            title: 'Caught Exception During Inquiry Questions Validation',
```

```
            status: 'error',
        }]);
    });

});
```

Troubleshoot Custom Scripts for Life Sciences

To troubleshoot and debug custom scripts for Life Sciences Cloud for Customer Engagement workflow validation, use Chrome Developer Tools. The Console tab shows log information, while the Network tab helps monitor network requests, responses, and performance.

REQUIRED EDITIONS

Available in: Lightning Experience
Available in: **Enterprise** and **Unlimited** Editions with Life Sciences Cloud, Life Sciences Cloud for Customer Engagement Add-on license, and the Life Sciences Customer Engagement managed package.

Troubleshoot in the Console Tab

Troubleshoot custom scripts from the Chrome Developer Tools Console tab.

1. Open Chrome Developer Tools.
2. Go to the Console tab.
3. Review the output for custom scripts.
   Custom scripts can use these messages to provide insights into script execution, variable values, and debugging information.
   - `console.log()`
   - `console.warn()`
   - `console.error()`
4. To investigate specific issues, filter the logs by type.
   For example, filter by Error, Warning, or Info.

Troubleshoot Network Requests and Responses

Troubleshoot custom scripts from the Chrome Developer Tools Networks tab. When a custom script makes asynchronous calls, you can monitor these requests in the Network tab.

1. Open Chrome Developer Tools.
2. Go to the Network tab.
3. To monitor network activity, look for requests that are related to your custom script and how it queries or retrieves data.

4. To inspect requests and responses, click a specific network request and review its details, including the headers, payload, and response.
5. To show data that's returned by the server, select the Response tab from within a network request. The data that's returned by the server can help you debug issues that are related to data retrieval.

Example Visit Action Validation Script

Visit Action Validation Custom Scripts help make sure that users follow business rules before signing or submitting a visit. These scripts run when the user selects the Sign or Submit action.

REQUIRED EDITIONS

Available in: Lightning Experience
Available in: **Enterprise** and **Unlimited** Editions with Life Sciences Cloud, Life Sciences Cloud for Customer Engagement Add-on license, and the Life Sciences Customer Engagement managed package.

To create a custom script for Visit Action Validation, create a Lightning component with these attributes:

- For the Script Name, use `visitSampleScript`.
- For the component name, use the Lightning component name, for example, *visitSampleScript*.
- For the Type, use `Visit Action Validation`.

In the Lightning component's JavaScript file, add the custom script from the example later and deploy the Lightning component to your org. The custom script performs validation whenever someone submits the Visit Action and when it's signed.

Be aware:

- If you ever change the Lightning component and save it, also find the record in Custom Scripts and click Refresh.
- If you create more than one script with Visit Action validation as a type, only the first version of the script runs. Which script runs is based on ID or creation date.
- The same Visit Action validation script runs in the mobile and Web versions of the app so be sure to test on both.

The example script contains six validation rules:

Validation Rules

Rule	Use	Triggers	Error Message	On Success
atLeastOneSampleIsRequired	Ensures that the visit contains at least one sample (ProductDisburse	Always runs	At least one sample must be added to the visit.	Shows the number of samples for the visit

Rule	Use	Triggers	Error Message	On Success
	ment).			
atLeastOneDetailAndSampleAreRequired	Requires both a sample and detailed product (ProviderVisitProdDetailing).	Always runs	At least one sample and detailed product must be added to the visit.	Shows counts of both samples and detailed products
atLeastOneMessageIsRequiredForEachVisitDetail	Each detailed product must have at least one message.	Only when the user is a Field Sales Representative (specific profile) or when the Visit channel is set to In-Person	At least one message is required for each detailed product when the channel is 'In-Person' and the user has a 'Field Sales Representative' profile	Skips other profiles or non-in-person visits
specificSampleDependencyCheck	If a specific product is selected, the rule checks if other required products are also selected.	When samples exist	If *<product1>* is added to a visit, *<product2>* must also be added.	N/A
isAtLeastOneHCP	HCO (Healthcare Organization or Institution) calls must have at least one HCP (Healthcare Provider) account attendee.	Only for Institution Accounts	At least one HCP (Healthcare Professional) must be associated when creating a visit for an HCO (Healthcare Organization).	Skips if the primary account is a Person Account (HCP)
isMoreThanOneHCO	Restricts to only one HCO (Institution) attendee per call.	Always runs	Only 1 HCO (Healthcare Organization) attendee can be added per visit.	Shows a count of HCO accounts

The sample script also contains three reusable helper methods:

- *getActionName(env)*
  - Gets the action name from the environment
  - Determines whether validation should run

  - Only runs for Submit, Sign, and runCustomScriptValidations
- *parseContextData(record)*
  - Safely extracts context data from the record object
  - Handles both JSON strings and objects (including Proxy objects)
  - Returns an empty object on error
- *getFieldData(contextData, baseFieldName)*
  - This is a platform-aware field data retrieval method.
  - For web, it uses *FieldName.VisitId* (for example, `ProductDisbursement.VisitId`.)
  - For mobile, it uses a simple field name (for example, `ProductDisbursement`.)
  - The method automatically tries both and returns whichever exists.

There are several aspects to be aware of and handle properly when you implement the script:

- On mobile and web, when the Visit Action Validation script runs, the data for the Visit may only be in memory, so you need to access the current data by processing the fields in the contextData. You can use db.query but that should be reserved for data which is not being modified in the Visit, such as checking the record type of an Account.
- There could be differences in the context data passed to the custom script between mobile and web. The example custom script handles these differences by using the common helper method `getFieldData`.

Example

```
(() => {
   // Note: Old data extraction functions removed - no longer needed
   // since businessRuleValidator provides clean, structured parameters


   // Platform detection flag - will be set in entry point
   let hasWebField = false;


   /**
    * Gets the action name from environment
    * @param {Object} env - JsEnv object for environment options
    * @returns {string} The action name or empty string
    */
   function getActionName(env) {
       try {
           if (env && typeof env.getOption === 'function') {
               return env.getOption('actionName') || '';
           }
           return '';
```

```
        } catch (error) {
            return '';
        }
    }


    // AccountDAO - Data Access Object for account-related operations
    var accountDao = (function () {
        var instance;
        var currentRecord;
        var isPersonAccount;
        var isInstitution;
        var childCallAccounts;
        var accountCache = new Map(); // Simple cache for account data


        // Helper functions for accountDao - moved inside closure to access
currentRecord
        async function checkForPersonAccount() {
            let accountId = currentRecord.stringValue("AccountId");

            // If not found, try extracting directly from context data
            if (!accountId) {
                try {
                    const contextData = parseContextData(currentRecord);

                    // Try different possible locations for AccountId
                    accountId = contextData.ProviderVisit?.AccountId ||
                                 contextData.Visit?.AccountId ||
                                contextData.AccountId ||
                                contextData.Account?.Id ||
                                contextData.Account;
                } catch (e) {
                    // Error accessing context data, continue with null acc
ountId
                    accountId = null;
                }
            }

            if (!accountId) {
                return false; // Default to not person account if no accoun
t ID
            }
```

```
            try {
                let account = await selectAccountById(accountId);
                let result = account && account.length > 0 ? account[0].boo
lValue("IsPersonAccount") : false;
                return result;
            } catch (error) {
                return false; // Default to false on error
            }
        }


        async function checkForInstitution() {
            let accountId = currentRecord.stringValue("AccountId");
            // If not found, try extracting directly from context data
            if (!accountId) {
                try {
                    const contextData = parseContextData(currentRecord);

                    // Try different possible locations for AccountId
                    accountId = contextData.ProviderVisit?.AccountId ||
                              contextData.Visit?.AccountId||
                               contextData.AccountId ||
                               contextData.Account?.Id ||
                               contextData.Account;
                } catch (e) {
                    // Error accessing context data, continue with null acc
ountId
                }
            }

            if (!accountId) {
                return false; // Default to not institution if no account I
D
            }

            try {
                let account = await selectAccountById(accountId);
                let isPersonAccount = account && account.length > 0 ? accou
nt[0].boolValue("IsPersonAccount") : false;

                // If it's not a Person Account, then it's a Business Accou
nt (HCO)
```

```
                let result = !isPersonAccount;
                return result;
            } catch (error) {
                return false; // Default to false on error
            }
        }


        async function selectChildCallAccountsById() {
            // Extract attendee account IDs directly from the JSON data
            // The attendee data is in the Visit.ParentVisitId array in the
JSON

            // Get the data from the current record context
            let contextData;
            try {
                contextData = parseContextData(currentRecord);
            } catch (error) {
                return [];
            }

            // Extract attendee account IDs from Visit.ParentVisitId array
            const attendeeVisits = contextData?.["Visit.ParentVisitId"] ||
contextData?.["ChildVisit"];

            if (!Array.isArray(attendeeVisits) || attendeeVisits.length ===
0) {
                return [];
            }

            // Extract AccountIds from the attendee visits
            const attendeeAccountIds = attendeeVisits
                .map(visit => visit.AccountId || visit.accountid)  // Try P
ascalCase first, then lowercase
                .filter(accountId => accountId);

            if (attendeeAccountIds.length === 0) {
                return [];
            }


            // Query the Account records for these IDs
            let result = await db.query(
```

```
            "Account",
            await new ConditionBuilder(
                "Account",
                new SetCondition("Id", "IN", attendeeAccountIds)
            ).build(),
            ["Id", "Name", "IsPersonAccount"]
        );

        return result || [];
    }


    function getRecordId(record) {
        return record ? record.stringValue("Id") : null;
    }


    async function selectAccountById(accountId) {
        // Check cache first
        if (accountCache.has(accountId)) {
            return accountCache.get(accountId);
        }

        // If not in cache, query database
        let accounts = await db.query(
            "Account",
            await new ConditionBuilder(
                "Account",
                new FieldCondition("Id", "=", accountId)
            ).build(),
            ["Id", "Name", "IsPersonAccount"]
        );

        // Cache the result
        accountCache.set(accountId, accounts);

        return accounts;
    }


    var initialize = async function(record) {
        currentRecord = record;
        // Clear cache for new record context
```

```
        accountCache.clear();
        isPersonAccount = await checkForPersonAccount();
        isInstitution = await checkForInstitution();
        childCallAccounts = await selectChildCallAccountsById();
    };


    var getIsPersonAccount = function () {
        return isPersonAccount;
    };


    var getIsInstitution = function () {
        return isInstitution;
    };


    var getChildCallAccounts = function () {
        return childCallAccounts;
    };


    var createInstance = function () {
        return {
            initialize: initialize,
            getIsPersonAccount: getIsPersonAccount,
            getChildCallAccounts: getChildCallAccounts,
            getIsInstitution: getIsInstitution,
        };
    };


    return {
        getInstance: function () {
            return instance || (instance = createInstance());
        },
    };
})();


// Main function that businessRuleValidator calls
async function validateVisit() {
    try {
```

```
            if (!record) {
                return [{
                    title: "Error in validation",
                    status: "error",
                    error: "No record provided"
                }];
            }

            // Use the properly provided business rule parameters from oute
r scope
            const validationResults = await runValidation();

            // Platform-specific handling: Web uses Promise.all(), Mobile d
oesn't
            let resolvedResults;
            if (hasWebField) {
                // Web platform - use Promise.all() to resolve all promises
                resolvedResults = await Promise.all(validationResults);
            } else {
                // Mobile platform - use validationResults directly
                resolvedResults = validationResults;
            }

            // Ensure we always return an array
            const finalResults = Array.isArray(resolvedResults) ? resolvedR
esults : [resolvedResults];
            return finalResults;
        } catch (error) {
            return [{
                title: "Error in validation",
                status: "error",
                error: error.message
            }];
        }
    }

    // Function to run the validation with data from outer scope
    async function runValidation() {


        // Initialize accountDao with the proper record object (JsDbObject)
        await accountDao.getInstance().initialize(record);
```

```
        // Always validate in this version
        const isValidationRequired = true;


        // Only run validations if needed
        let validationResults = [];
        if (isValidationRequired) {
            // Array of validation functions to run
            // Add new validation functions to this array
            const validationFunctions = [
                atLeastOneSampleIsRequired,
                atLeastOneDetailAndSampleAreRequired,
                atLeastOneMessageIsRequiredForEachVisitDetail,
                specificSampleDependencyCheck,
                isAtLeastOneHCP,
                isMoreThanOneHCO,
                // Add new validation functions here one at a time
                // Example: validateSampleType,
                // Example: validateComplianceAgreement,
            ];

            // Run all validation functions (handling both sync and async)

            validationResults = validationFunctions.map((validationFn, inde
x) => {
                try {
                    // Call validation functions - they access record, use
r, db, env from outer scope
                    const result = validationFn();
                // If the result is a Promise, return it as is for Promis
e.all
                if (result && typeof result.then === 'function') {
                    return result.then(asyncResult => {
                        return asyncResult;
                    }).catch(error => {
                        return {
                            title: `Error in ${validationFn.name}: ${erro
r.message}`,
                            status: "error",
                            error: error.message
                        };
                    });
```

```
                }
                return result;
            } catch (error) {
                return {
                    title: `Error in ${validationFn.name}: ${error.messag
e}`,
                    status: "error",
                    error: error.message
                };
            }
            });


            } else {
                // Default return when validation is not required
                validationResults = [{
                    title: "Validation not required",
                    status: "success"
                }];
            }
        return validationResults;
    }





    // Helper function to get context data safely
    function parseContextData(record) {
        try {
            if (!record || typeof record.getContextData !== 'function') {
                return {};
            }

            const contextData = record.getContextData();

            // Handle different return types from getContextData()
            if (typeof contextData === 'string') {
                // If it's a JSON string, parse it
                return JSON.parse(contextData);
            } else if (typeof contextData === 'object' && contextData !== n
ull) {
                // If it's already an object (including Proxy), use it dire
```

```
ctly
                return contextData;
            } else {
                return {};
            }
        } catch (error) {
            return {};
        }
    }

    /**
     * Helper function to get field data with web/mobile fallback
     * Web uses nested field paths (e.g., "ObjectName.VisitId")
     * Mobile uses simple field names (e.g., "ObjectName")
     *
     * @param {Object} contextData - The context data object
     * @param {string} baseFieldName - The base field name (e.g., "ProductD
isbursement")
     * @returns {*} The field data from web or mobile field, or undefined
     */
    function getFieldData(contextData, baseFieldName) {
        const webField = `${baseFieldName}.VisitId`;
        const mobileField = baseFieldName;
        return contextData?.[webField] || contextData?.[mobileField];
    }

    // Validation rule: at least one sample is required
    function atLeastOneSampleIsRequired() {
        let hasSamples = false;
        let sampleCount = 0;

        try {
            // Get context data from the record object (from outer scope)
            const contextData = parseContextData(record);
            // Use helper to get field data with web/mobile fallback
            const sampleData = getFieldData(contextData, "ProductDisburseme
nt") || null;

            // Handle Proxy arrays properly
            if (sampleData) {
                try {
                    // Try to get length property (works for both arrays an
d Proxy arrays)
```

```
                    sampleCount = sampleData.length || 0;
                    hasSamples = sampleCount > 0;


                } catch (lengthError) {
                    // Handle Proxy length access error
                    // Fallback: check if object has any enumerable propert
ies
                    try {
                        const keys = Object.keys(sampleData);
                        hasSamples = keys.length > 0;
                        sampleCount = keys.length;
                    } catch (keysError) {
                        // Error getting keys - graceful fallback
                        hasSamples = false;
                        sampleCount = 0;
                    }
                }
            }
        } catch (e) {
            // Handle validation error gracefully
            hasSamples = false;
            sampleCount = 0;
        }

        return {
            title: hasSamples ?
                `Found ${sampleCount} sample(s)` :
                "At least one sample must be added to the visit.",
            status: hasSamples ? "success" : "error"
        };
    }


    // Validation rule: at least one detail and sample are required
    function atLeastOneDetailAndSampleAreRequired() {
        try {
            const contextData = parseContextData(record);

            // Use helper to get field data with web/mobile fallback
            const productDisbursementData = getFieldData(contextData, "Prod
uctDisbursement");
            const providerVisitProdDetailingData = getFieldData(contextDat
```

```
a, "ProviderVisitProdDetailing");

            // Handle Proxy arrays properly for both fields
            let sampleCount = 0;
            let detailCount = 0;
            let hasProductDisbursement = false;
            let hasProviderVisitProdDetailing = false;

            // Check product disbursement
            if (productDisbursementData) {
                try {
                    sampleCount = productDisbursementData.length || 0;
                    hasProductDisbursement = sampleCount > 0;
                } catch (e) {
                    // Error accessing length property
                    const keys = Object.keys(productDisbursementData ||
{});
                    sampleCount = keys.length;
                    hasProductDisbursement = sampleCount > 0;
                }
            }

            // Check provider visit prod detailing
            if (providerVisitProdDetailingData) {
                try {
                    detailCount = providerVisitProdDetailingData.length ||
0;
                    hasProviderVisitProdDetailing = detailCount > 0;
                } catch (e) {
                    // Error accessing length property - try Object.keys fa
llback
                    const keys = Object.keys(providerVisitProdDetailingDat
a);
                    detailCount = keys.length;
                    hasProviderVisitProdDetailing = detailCount > 0;
                }
            }

            if (hasProductDisbursement && hasProviderVisitProdDetailing) {
                return {
                    title: `Found ${sampleCount} sample(s) and ${detailCoun
t} detailed product(s)`,
                    status: "success"
```

```
                };
            }

            return {
                title: "At least one sample and detailed product must be ad
ded to the visit.",
                status: "error"
            };
        } catch (e) {
            return {
                title: "At least one sample and detailed product must be ad
ded to the visit.",
                status: "error",
                error: e.message
            };
        }
    }


    // Validation rule: at least one message is required for each visit det
ail
    async function atLeastOneMessageIsRequiredForEachVisitDetail() {
        try {
            let userId;

            // Try to get user Id from user (from outer scope)
            if (user) {
                try {
                    userId = user.stringValue('Id');
                } catch (error) {
                    // Try alternative access methods
                    userId = user.Id || user["Id"];
                    if (!userId) {
                        // Error accessing userId, continue with fallback
                    }
                }
            }

            if (!userId) {
                return {
                    title: 'Profile validation skipped - no userId availabl
e',
                    status: "success",
```

```
                };
            }


            // As rep user don't have access to User.ProfileId and Profile
table
            // we can only check by ProfileIdentifier in UserAdditionalInfo
            //Please replace the Profile_Id with the id of the profile you
want to apply this validation
            let targetProfileId = 'Profile_Id';
            let userAdditionalInfoResults;
            let isFieldSalesRep = false;

            try {
                userAdditionalInfoResults = await db.query(
                    "UserAdditionalInfo",
                    await new ConditionBuilder(
                        "UserAdditionalInfo",
                        new FieldCondition("UserId", "=", userId)
                    ).build(),
                    ["Id", "ProfileIdentifier"]
                );


                if (userAdditionalInfoResults && userAdditionalInfoResult
s.length > 0) {
                    const profileId = userAdditionalInfoResults[0].stringVa
lue('ProfileIdentifier');
                    isFieldSalesRep = profileId === targetProfileId;
                } else {
                    return {
                        title: 'Profile validation skipped - profile not fo
und',
                        status: "success",
                    };
                }

            } catch (error) {
                return {
                    title: 'Profile validation skipped - unable to query pr
ofile',
                    status: "success",
                };
```

```
            }


            if (!isFieldSalesRep) {
                return {
                    title: `Profile validation skipped - user is not Field
Sales Representative`,
                    status: "success",
                };
            }


            // Get visit context data from the record object
            const visitData = parseContextData(record);


            // Check if channel is "In-Person"
            const visitChannel = visitData?.Visit?.channel || visitData?.Pr
oviderVisit?.Channel || '';


            if (visitChannel !== "In-Person") {
                return {
                    title: `Message validation skipped - visit channel is
"${visitChannel}", not "In-Person"`,
                    status: "success",
                };
            }


            // Check if we have visit details to validate
            // Use helper to get field data with web/mobile fallback
            const visitDetails = getFieldData(visitData, "ProviderVisitProd
Detailing");


            if (!Array.isArray(visitDetails) || visitDetails.length === 0)
{
                return {
                    title: 'Message validation passed - no visit details to
validate',
                    status: "success"
                };
```

```
            }


            // Validate each visit detail has at least one message
            let detailsWithoutMessages = [];


            visitDetails.forEach((detail, index) => {
                // Use helper to get field data with web/mobile fallback
                const messages = getFieldData(detail, "ProviderVisitDtlProd
uctMsg");
                const hasMessages = Array.isArray(messages) && messages.len
gth > 0;


                if (!hasMessages) {
                    const detailInfo = {
                        index: index + 1,
                        productId: detail?.productid || 'Unknown Product',
                        uid: detail?.uid || 'Unknown Detail'
                    };
                    detailsWithoutMessages.push(detailInfo);
                }
            });


            // Set validation result
            if (detailsWithoutMessages.length > 0) {
                return {
                    title: "At least one message is required for each detai
led product when the channel is 'In-Person' and the user has a 'Field Sale
s Representative' profile.",
                    status: "error"
                };
            } else {
                return {
                    title: `All ${visitDetails.length} detailed products ha
ve messages - Field Sales Rep In-Person validation passed`,
                    status: "success"
                };
            }
```

```
        } catch (error) {
            return {
                title: "At least one message is required for each detailed
product when the channel is 'In-Person' and the user has a 'Field Sales Re
presentative' profile.",
                status: "error",
                error: error.message
            };
        }
    }


    /**
     * The rule 'specificSampleDependencyCheck' blocks the user from submit
ting a visit.
     * Validation: If sample "Immunexis 10mg" is selected,
     * then "ADRAVIL Sample Pack 5mg" must also be selected.
     * @returns result { title: string, status: "success" | "error" };
     */
    async function specificSampleDependencyCheck() {


        try {
            // Get visit context data from record (from outer scope)
            let visitData = parseContextData(record);

            // Check if we have samples to validate
            // Use helper to get field data with web/mobile fallback
            let samples = getFieldData(visitData, "ProductDisbursement");

            // Handle Proxy arrays properly
            let samplesCount = 0;
            let isValidSamples = false;

            if (samples) {
                try {
                    samplesCount = samples.length || 0;
                    isValidSamples = samplesCount > 0;
                } catch (e) {
                    // Error accessing samples length
                    const keys = Object.keys(samples || {});
                    samplesCount = keys.length;
                    isValidSamples = samplesCount > 0;
```

```
                }
            }

            if (!isValidSamples) {
                return {
                    title: 'Sample dependency validation passed - no sample
s to validate',
                    status: "success"
                };
            }


            // Get all product item IDs from samples
            let productItemIds = [];
            try {
                if (samples && typeof samples === 'object') {
                    // Handle both array and Proxy array
                    for (let i = 0; i < samplesCount; i++) {
                        try {
                            const sample = samples[i];


                            if (sample) {
                                const productItemId = sample.ProductItemId
|| sample.productitemid
                                productItemIds.push(productItemId);
                            }
                        } catch (sampleError) {
                            // Error accessing sample
                        }
                    }
                }
            } catch (mappingError) {
                // Error mapping product item IDs
            }



            if (productItemIds.length === 0) {
                return {
                    title: 'Sample dependency validation passed - no produc
t item IDs found',
                    status: "success"
```

```
                };
            }

            // Query ProductItem to get Product2Id
            let productItems = await db.query(
                   "ProductItem",
                   await new ConditionBuilder(
                     "ProductItem",
                     new SetCondition("Id", "IN", productItemIds)
                   ).build(),
                   ["Id", "Product2Id"]
                 );


            // Extract Product2Ids and create a map of productItemId to Pro
duct2Id
            let product2Ids = [];
            let productItemToProduct2Map = new Map();

            if (productItems && Array.isArray(productItems)) {
                productItems.forEach(item => {
                    const productItemId = item.stringValue("Id");
                    const product2Id = item.stringValue("Product2Id");

                    if (product2Id) {
                        product2Ids.push(product2Id);
                        productItemToProduct2Map.set(productItemId, product
2Id);
                    }
                });
            }

            // Query Product2 to get product names
            let product2Items = await db.query(
                   "Product2",
                   await new ConditionBuilder(
                     "Product2",
                     new SetCondition("Id", "IN", product2Ids)
                   ).build(),
                   ["Id", "Name"]
                 );

            // Create a map of Product2Id to Name
```

```
            let product2NameMap = new Map();
            if (product2Items && Array.isArray(product2Items)) {
                product2Items.forEach(item => {
                    const id = item.stringValue("Id");
                    const name = item.stringValue("Name");
                    product2NameMap.set(id, name);
                });
            }

            // Create a map of productItemId to ProductName (via Product2)
            let itemToProductNameMap = new Map();
            productItemToProduct2Map.forEach((product2Id, productItemId) =>
{
                const productName = product2NameMap.get(product2Id);
                if (productName) {
                    itemToProductNameMap.set(productItemId, productName);
                }
            });

            // Get all sample names for the current visit
            let sampleNames = [];
            try {
                if (samples && typeof samples === 'object') {
                    // Handle both array and Proxy array for sample names
                    for (let i = 0; i < samplesCount; i++) {
                        try {
                            const sample = samples[i];
                            if (sample) {
                                const productItemId = sample.ProductItemId
|| sample.productitemid;
                                const productName = itemToProductNameMap.ge
t(productItemId) || '';
                                if (productName) {
                                    sampleNames.push(productName);
                                }
                            }
                        } catch (sampleError) {
                            // Error accessing sample for name mapping
                        }
                    }
                }
            } catch (nameMappingError) {
                // Error mapping sample names
```

```
            }

            // Check if Immunexis 10mg is present
            const targetSample = "Immunexis 10mg";
            const requiredSample = "ADRAVIL Sample Pack 5mg";
            let hasImmunexis = sampleNames.includes(targetSample);


            if (hasImmunexis) {
                // If Immunexis is present, check if ADRAVIL is also presen
t
                let hasAdravil = sampleNames.includes(requiredSample);


                if (!hasAdravil) {
                    return {
                        title: "If Immunexis 10mg is added to a visit, ADRA
VIL Sample Pack 5mg must also be added. However, ADRAVIL Sample Pack 5mg c
an be added without Immunexis 10mg.",
                        status: "error"
                    };
                } else {
                    return {
                        title: "Sample dependency validation passed - both
Immunexis 10mg and ADRAVIL Sample Pack 5mg present",
                        status: "success"
                    };
                }
            } else {
                return {
                    title: "Sample dependency validation passed - no Immune
xis 10mg found",
                    status: "success"
                };
            }


      } catch (error) {
            // Error in specificSampleDependencyCheck
            // In case of database error, we might want to pass validation
or handle differently
            // For now, we'll pass the validation to avoid blocking the use
r due to technical issues
```

```
            return {
                title: "Sample dependency validation passed - technical err
or occurred",
                status: "success"
            };
        }
    }


    /**
     * The rule 'isAtLeastOneHCP' blocks the user from submitting a call.
     * Validation: Require at least one HCP (Person Account) for a HCO (Ins
titution Account) call on Submit.
     * @returns result { title: string, status: "success" | "error" };
     */
    async function isAtLeastOneHCP() {
        try {

            // Log current account details from record (from outer scope)
            const currentAccountId = record.stringValue("AccountId");


            // Use accountDao to check if current account is a Person Accou
nt
            let isPersonAccount = await accountDao.getInstance().getIsPerso
nAccount();
            let isInstitution = await accountDao.getInstance().getIsInstitu
tion();


            if (isPersonAccount) {
                // This is already a Person Account (HCP), so requirement i
s met
                return {
                    title: "HCP validation passed - current account is a Pe
rson Account (HCP)",
                    status: "success"
                };
            }


            // Only apply HCP validation to Institution accounts
            if (!isInstitution) {
```

```
                return {
                    title: "HCP validation skipped - account is not an Inst
itution Account",
                    status: "success"
                };
            }


            // This is an Institution Account, check for HCP attendees
            let childCallAccounts = await accountDao.getInstance().getChild
CallAccounts();


            let isRequirementValid = false;
            let hcpAttendees = [];
            let nonHcpAttendees = [];


            if (Array.isArray(childCallAccounts) && childCallAccounts.lengt
h > 0) {
                // Check if any attendee is a Person Account (HCP)
                for (let i = 0; i < childCallAccounts.length; i++) {
                    let attendee = childCallAccounts[i];
                    let attendeeIsPersonAccount = attendee.boolValue("IsPer
sonAccount");
                    let attendeeName = attendee.stringValue("Name") || atte
ndee.stringValue("Id");

                    if (attendeeIsPersonAccount) {
                        isRequirementValid = true;
                        hcpAttendees.push(attendeeName);
                    } else {
                        nonHcpAttendees.push(attendeeName);
                    }
                }
            }




            // Add more descriptive message based on the scenario
            if (!isRequirementValid) {
                return {
```

```
                    title: "At least one HCP (Healthcare Professional) must
be associated when creating a visit for an HCO (Healthcare Organizatio
n).",
                    status: "error"
                };
            } else {
                return {
                    title: `HCP validation passed - Institution Account wit
h ${hcpAttendees.length} HCP attendee(s): ${hcpAttendees.join(', ')}`,
                    status: "success"
                };
            }


     } catch (error) {
            // Error in isAtLeastOneHCP
            // In case of error, fail the validation to be safe
            return {
                title: "HCP validation failed - error occurred during valid
ation",
                status: "error",
                error: error.message
            };
       }
   }


   /**
    * The rule 'isMoreThanOneHCO' blocks user from submitting a call.
    * Validation: Restrict to one HCO (Institution Account) attendee per C
all.
    * @returns result { title: string, status: "success" | "error" };
    * Note: Expected only 1 HCO attendee per call)
    */
   async function isMoreThanOneHCO() {
            try {

                let counter = 0;
                let isPersonAccount = await accountDao.getInstance().getIsP
ersonAccount();
                let accsRelatedToChildCall = await accountDao.getInstanc
e().getChildCallAccounts();
```

```
                let hcoAccounts = [];
                let hcpAccounts = [];


                if (isPersonAccount || accsRelatedToChildCall.length) {
                    for (let i = 0; i < accsRelatedToChildCall.length; i++)
{
                        let relatedAcc = accsRelatedToChildCall[i];
                        let attendeeIsPersonAccount = relatedAcc.boolValu
e("IsPersonAccount");
                        let attendeeName = relatedAcc.stringValue("Name")
|| relatedAcc.stringValue("Id");

                        if (!attendeeIsPersonAccount) {
                            counter++;
                            hcoAccounts.push(attendeeName);
                        } else {
                            hcpAccounts.push(attendeeName);
                        }
                    }
                } else {
                    counter++;
                }


                const isValid = counter <= 1;


                if (!isValid) {
                    return {
                        title: "Only 1 HCO (Healthcare Organization) attend
ee can be added per visit.",
                        status: "error"
                    };
                } else {
                    return {
                        title: `HCO count validation passed - found ${count
er} HCO account(s)`,
                        status: "success"
                    };
                }
```

```
        } catch (error) {
                // Error in isMoreThanOneHCO
                return {
                    title: "HCO count validation failed - error occurred du
ring validation",
                    status: "error",
                    error: error.message
                };
            }
        }




    // Entry point: Check if proper parameters are provided
    if (record && user && env && db) {
        // Check action name - only run validation for specific actions
        const actionName = getActionName(env);
        const allowedActions = ['Submit', 'Sign', 'runCustomScriptValidatio
ns'];

        if (!allowedActions.includes(actionName)) {
            // Skip validation for other actions
            return [{
                title: `Validation skipped - action is "${actionName}"`,
                status: 'success'
            }];
        }

        // Platform detection: Check which field structure exists to determ
ine wrapping behavior
        const contextData = parseContextData(record);

        // Web uses nested field paths (e.g., "ProviderVisit")
        // Mobile uses simple field names (e.g., "Visit")
        hasWebField = contextData?.["ProviderVisit"] !== undefined;

        // Wrap in array if using web platform, don't wrap for mobile
        const wrapAsArray = hasWebField;

        // Apply wrapping based on platform
        if (wrapAsArray) {
```

```
            // Web solution - wrap in array
            return [validateVisit()];
        } else {
            // Mobile solution - no wrapping
            return validateVisit();
        }
    }

    // If no parameters provided yet, return validation function wrapped in
array
    return [validateVisit];
 })();
```

Life Sciences Custom Scripts Reference

Understand the available JavaScript classes, functions, and variables that you can include in a Lightning web component for custom scripts in Life Sciences Cloud for Customer Engagement.

REQUIRED EDITIONS

Available in: Lightning Experience
Available in: **Enterprise** and **Unlimited** Editions with Life Sciences Cloud, Life Sciences Cloud for Customer Engagement Add-on license, and the Life Sciences Customer Engagement managed package.

Classes for Life Sciences Custom Scripts

Life Sciences Cloud for Customer Engagement supports various JavaScript classes in custom scripts. These classes enable you to create powerful and flexible logic within the script, particularly for constructing queries and managing data. Each class includes a description of its purpose, constructor details, parameters, methods, and examples.

Variables for Life Sciences Custom Scripts

Global variables in Life Sciences Cloud for Customer Engagement custom scripts provide access to contextual information such as environment details, database interactions, current record data, and user information. Use these variables to develop dynamic, data-driven custom scripts.

Functions for Life Sciences Custom Scripts

Use functions to enhance the robustness and adaptability of your custom scripts in Life Sciences Cloud for Customer Engagement. These functions provide specific utilities, from error handling and namespace management to retrieving metadata about available fields.

Classes for Life Sciences Custom Scripts

Life Sciences Cloud for Customer Engagement supports various JavaScript classes in custom scripts. These classes enable you to create powerful and flexible logic within the script, particularly for constructing queries and managing data. Each class includes a description of its purpose, constructor details, parameters, methods, and examples.

REQUIRED EDITIONS

Available in: Lightning Experience
Available in: **Enterprise** and **Unlimited** Editions with Life Sciences Cloud, Life Sciences Cloud for Customer Engagement Add-on license, and the Life Sciences Customer Engagement managed package.

AndCondition Class
Builds a SOQL condition based on multiple nested conditions. Extends GroupCondition.

Condition Class
An abstract class that serves as the foundation for all conditions and lacks a concrete implementation.

ConditionBuilder Class
Builds a WHERE clause to use as part of a SOQL query. Validates if the current user has access to the fields that are used in conditions.

ConditionEnhancedBuilder Class
Serves the same purpose as the ConditionBuilder class. Only use the ConditionEnhancedBuilder class with the db.bulkQuery function. The ConditionEnhancedBuilder class doesn't validate whether the current user has access to the fields that are used in the query conditions.

DateFieldCondition Class
Builds a SOQL condition for a date field based on the specified operator. Extends FieldCondition.

DateTimeFieldCondition Class
Builds a SOQL condition for a datetime field based on the specified operator. Extends FieldCondition.

FieldCondition Class
Builds a SOQL condition for a field based on the specified operator. Extends OperatorCondition.

GroupCondition Class
Builds a SOQL condition based on multiple nested conditions. Extends Condition.

OperatorCondition Class
Extends the Condition class. Used as a super class for specific conditions.

OrCondition Class
Builds a SOQL condition based on multiple nested conditions. Extends GroupCondition.

Query Class
Creates a SOQL query based on an input. Only use the Query class as a subquery of SetCondition. To build a complex WHERE clause, pass SetCondition to ConditionBuilder.

SetCondition Class
Builds a SOQL condition for a field whose value is in a specified range. Extends OperatorCondition.

AndCondition Class

Builds a SOQL condition based on multiple nested conditions. Extends GroupCondition.

REQUIRED EDITIONS

Available in: Lightning Experience
Available in: **Enterprise** and **Unlimited** Editions with Life Sciences Cloud, Life Sciences Cloud for Customer Engagement Add-on license, and the Life Sciences Customer Engagement managed package.

constructor()

Builds a GroupCondition with **AND** as the nesting operator.

toSoql()

Returns the generated SOQL query as a string.

Example

```
await new ConditionBuilder(
    "HealthcareProvider",
    new AndCondition()
        .add(new FieldCondition("Status", "=", "Active"))
        .add(new FieldCondition("IsPrimaryProvider", "=", true))
).build()

// Returns: "(Status = \'Active\') AND (IsPrimaryProvider = true)"
```

Condition Class

An abstract class that serves as the foundation for all conditions and lacks a concrete implementation.

REQUIRED EDITIONS

Available in: Lightning Experience
Available in: **Enterprise** and **Unlimited** Editions with Life Sciences Cloud, Life Sciences Cloud for Customer Engagement Add-on license, and the Life Sciences Customer Engagement managed package.

Important Don't use the Condition class directly. Instead, use the predefined classes that extend the Condition class. Provide an instance of a Condition subclass as a parameter for the ConditionBuilder. The ConditionBuilder then uses the Condition subclass to construct the WHERE clause part of the query.

getRequiredFields()

Returns an empty object.

toSoql()

Returns an empty string.

ConditionBuilder Class

Builds a WHERE clause to use as part of a SOQL query. Validates if the current user has access to the fields that are used in conditions.

REQUIRED EDITIONS

Available in: Lightning Experience
Available in: **Enterprise** and **Unlimited** Editions with Life Sciences Cloud, Life Sciences Cloud for Customer Engagement Add-on license, and the Life Sciences Customer Engagement managed package.

constructor(objectType, condition)

Creates an instance of ConditionBuilder.

This method accepts these parameters.

Parameter	Type	Description
objectType	String	The object for which the query is built.
condition	Condition	The condition to use for the query.

build()

Builds the condition based on the provided object and condition. Checks if the user has access to the fields that are used in conditions, and returns an exception if they don't have access.

Example

```
await new ConditionBuilder(
   "Account",
   new FieldCondition("Name", "=", "sForce")
).build()

// Returns: "Name = 'sForce'"
```

ConditionEnhancedBuilder Class

Serves the same purpose as the ConditionBuilder class. Only use the ConditionEnhancedBuilder class with the db.bulkQuery function. The ConditionEnhancedBuilder class doesn't validate whether the current user has access to the fields that are used in the query conditions.

REQUIRED EDITIONS

Available in: Lightning Experience
Available in: **Enterprise** and **Unlimited** Editions with Life Sciences Cloud, Life Sciences Cloud for Customer Engagement Add-on license, and the Life Sciences Customer Engagement managed package.

constructor(objectType, condition)

Creates an instance of ConditionEnhancedBuilder.

This method accepts these parameters.

Parameter	Type	Description
objectType	String	The object for which the query is built.
condition	Condition	The condition to use for the query.

build()

Returns a JavaScript object with a SOQL string and a map of fields that are listed in the condition for each JavaScript object.

Example

```
await new ConditionEnhancedBuilder(
   "Account",
   new FieldCondition("Name", "=", "sForce")
).build()

/**
 * Returns this JavaScript object:
 * {
 *     soqlString: "Name = 'sForce'",
 *     fieldNamesByObjectNames: {"Account": ["Name"]}
 * }
 */
```

DateFieldCondition Class

Builds a SOQL condition for a date field based on the specified operator. Extends FieldCondition.

REQUIRED EDITIONS

Available in: Lightning Experience
Available in: **Enterprise** and **Unlimited** Editions with Life Sciences Cloud, Life Sciences Cloud for Customer Engagement Add-on license, and the Life Sciences Customer Engagement managed package.

constructor(dateField, operator, dateValue)

Creates an instance of DateFieldCondition.

This method accepts these parameters.

Parameter	Type	Description
dateField	String	The name of the field to use as part of a condition.
operator	String	The operator to use as part of a condition.
dateValue	Date	The date to use as part of a condition.

toSoql()

Returns the generated SOQL query as a string.

Example

```
await new ConditionBuilder(
    "HealthcareProvider",
    new DateFieldCondition("InitialStartDate", "=", new Date())
).build()

// Returns: "InitialStartDate = 2024-08-12"
```

DateTimeFieldCondition Class

Builds a SOQL condition for a datetime field based on the specified operator. Extends FieldCondition.

REQUIRED EDITIONS

Available in: Lightning Experience
Available in: **Enterprise** and **Unlimited** Editions with Life Sciences Cloud, Life Sciences Cloud for Customer Engagement Add-on license, and the Life Sciences Customer Engagement managed package.

constructor(dateTimeField, operator, dateTimeValue)

Creates an instance of DateTimeFieldCondition.

This method accepts these parameters.

Parameter	Type	Description
dateTimeField	String	The name of the field to use as part of a condition.
operator	String	The operator to use as part of a condition.
dateTimeValue	Date	The date to use as part of a condition.

toSoql()

Returns the generated SOQL query as a string.

Example

```
await new ConditionBuilder(
    "Inquiry",
    new DateTimeFieldCondition("CreatedDate", "=", new Date())
    ).build()

    // Returns: "CreatedDate = 2024-08-12T12:47:55.594Z"
```

FieldCondition Class

Builds a SOQL condition for a field based on the specified operator. Extends OperatorCondition.

REQUIRED EDITIONS

Available in: Lightning Experience
Available in: **Enterprise** and **Unlimited** Editions with Life Sciences Cloud, Life Sciences Cloud for Customer Engagement Add-on license, and the Life Sciences Customer Engagement managed package.

constructor(field, operator, value)

Creates an instance of FieldCondition.

This method accepts these parameters.

Parameter	Type	Description
field	String	The name of the field to use as part of a condition.
operator	String	The operator to use as part of a condition.
value	String	The value to use as part of a condition. A null value is supported for the = and != operators.

toSoql()

Returns the generated SOQL query as a string.

Example

```
await new ConditionBuilder(
   "Inquiry",
   new FieldCondition("Type", "=", "MedicalInquiry")
).build()

// Returns: "Type = 'MedicalInquiry'"
```

GroupCondition Class

Builds a SOQL condition based on multiple nested conditions. Extends Condition.

REQUIRED EDITIONS

Available in: Lightning Experience
Available in: **Enterprise** and **Unlimited** Editions with Life Sciences Cloud, Life Sciences Cloud for Customer Engagement Add-on license, and the Life Sciences Customer Engagement managed package.

constructor(nestingOperator)

Creates an instance of GroupCondition.

This method accepts these parameters.

Parameter	Type	Description
nestingOperator	String	The operator to use between nested conditions. Supported operators are **AND** and **OR**.

add(condition)

Adds a nested condition to the group condition.

This method accepts these parameters.

Parameter	Type	Description
condition	Condition	The condition to add to the group condition.

getRequiredFields(objectType)

Returns the required fields for a specified object type.

This method accepts these parameters.

Parameter	Type	Description
objectType	String	The name of the object to get the required fields for.

toSoql()

Returns the generated SOQL query as a string.

Example

```
await new ConditionBuilder(
    "HealthcareProvider",
    new GroupCondition("AND")
        .add(new FieldCondition("Status", "=", "Active"))
        .add(new FieldCondition("IsPrimaryProvider", "=", true))
).build()

// Returns: "(Status = \'Active\') AND (IsPrimaryProvider = true)"
```

OperatorCondition Class

Extends the Condition class. Used as a super class for specific conditions.

REQUIRED EDITIONS

Available in: Lightning Experience
Available in: **Enterprise** and **Unlimited** Editions with Life Sciences Cloud, Life Sciences Cloud for Customer Engagement Add-on license, and the Life Sciences Customer Engagement managed package.

constructor(field, operator)

Creates an instance of OperatorCondition.

This method accepts these parameters.

Parameter	Type	Description
field	String	The name of the field to use as part of a condition.
operator	String	The operator to use as part of a condition.
value	Array or Query	An array of values to use as part of a condition, or an instance of a Query object that specifies the field values to query.

getRequiredFields(objectType)

Returns the required fields for a specified object type.

This method accepts these parameters.

Parameter	Type	Description
objectType	String	The name of the object to get the required fields for.

toSoql()

Returns the generated SOQL query as a string.

OrCondition Class

Builds a SOQL condition based on multiple nested conditions. Extends GroupCondition.

constructor()

Builds a GroupCondition with **OR** as the nesting operator.

toSoql()

Returns the generated SOQL query as a string.

Example

```
await new ConditionBuilder(
    "HealthcareProvider",
    new OrCondition()
        .add(new FieldCondition("Status", "=", "Active"))
        .add(new FieldCondition("IsPrimaryProvider", "=", true))
).build()

// Returns: "(Status = \'Active\') OR (IsPrimaryProvider = true)"
```

Query Class

Creates a SOQL query based on an input. Only use the Query class as a subquery of SetCondition. To build a complex WHERE clause, pass SetCondition to ConditionBuilder.

REQUIRED EDITIONS

Available in: Lightning Experience
Available in: **Enterprise** and **Unlimited** Editions with Life Sciences Cloud, Life Sciences Cloud for Customer Engagement Add-on license, and the Life Sciences Customer Engagement managed package.

constructor()

Creates an empty instance of Query.

select(field)

Specifies the field name to select. Only one field name can be selected. Returns an instance of Query.

This method accepts these parameters.

Parameter	Type	Description
field	String	The name of the field to select.

from(sObjectType)

Specifies the object to query. Returns an instance of Query.

This method accepts these parameters.

Parameter	Type	Description
sObjectType	String	The name of the object to query.

where(condition)

Specifies the where condition for the query. Returns an instance of Query.

This method accepts these parameters.

Parameter	Type	Description
condition	Condition	The condition to use as part of the query.

getRequiredFields()

Returns the required fields for a specified object type.

This method is supported only on desktop and doesn't work on the Life Sciences Cloud mobile app.

toSoql()

Returns the generated SOQL query as a string.

Example

```
await new ConditionBuilder(
    "Account",
    new SetCondition(
        "Id",
        "IN",
        new Query()
            .select("AccountId")
            .from("Visit")
            .where(new FieldCondition("Status", "=","Planned"))
    )
).build();

// Returns: "Id IN (SELECT AccountId FROM Visit WHERE Status = \'Planned\')"
```

SetCondition Class

Builds a SOQL condition for a field whose value is in a specified range. Extends OperatorCondition.

REQUIRED EDITIONS

Available in: Lightning Experience
Available in: **Enterprise** and **Unlimited** Editions with Life Sciences Cloud, Life Sciences Cloud for Customer Engagement Add-on license, and the Life Sciences Customer Engagement managed package.

constructor(field, operator, values)

Creates an instance of SetCondition.

This method accepts these parameters.

Parameter	Type	Description
field	String	The name of the field to use as part of a condition.
operator	String	The operator to use as part of a condition.
value	Array or Query	An array of values to use as part of a condition, or an instance of a Query object that specifies the field values to query.

getRequiredFields(objectType)

Returns the required fields for a specified object type. This method accepts these parameters.

Parameter	Type	Description
objectType	String	The name of the object to get the required fields for.

toSoql()

Returns the generated SOQL query as a string.

Example

```
await new ConditionBuilder(
   "Inquiry",
   new SetCondition(
       "Type",
       "IN",
       ["MedicalInquiry", "ProductInformation"],
   )
).build()

// Returns: "Type IN (\'MedicalInquiry\',\'ProductInformation\')"
```

Variables for Life Sciences Custom Scripts

Global variables in Life Sciences Cloud for Customer Engagement custom scripts provide access to contextual information such as environment details, database interactions, current record data, and user information. Use these variables to develop dynamic, data-driven custom scripts.

REQUIRED EDITIONS

Available in: Lightning Experience
Available in: **Enterprise** and **Unlimited** Editions with Life Sciences Cloud, Life Sciences Cloud for Customer Engagement Add-on license, and the Life Sciences Customer Engagement managed package.

Environment Variable
The environment (env) variable manages environment settings. You can retrieve or create environment option values, get log information, and format labels for localization.

Database Variable
The db object provides access to the database to query records. All db functions are executed synchronously on the Life Sciences Cloud mobile app and asynchronously on desktop. In order to query correctly, use await syntax.

Record Variable
The record variable represents the current record in the context of the custom script. You can retrieve string, number, boolean, or date field values, and you can set values for fields on the current record.

User Variable
The user variable represents the current user and provides methods to access and modify field values on the user record. You can retrieve string, number, boolean, or date field values, and you can set values for user record fields.

Environment Variable

The environment (env) variable manages environment settings. You can retrieve or create environment option values, get log information, and format labels for localization.

REQUIRED EDITIONS

Available in: Lightning Experience
Available in: **Enterprise** and **Unlimited** Editions with Life Sciences Cloud, Life Sciences Cloud for Customer Engagement Add-on license, and the Life Sciences Customer Engagement managed package.

getOption(name)

Returns the value of the environment option. These options are set by default.

- **fromStatus**:
  - On desktop, fromStatus is set to the current value of a controlling field for the workflow path. This is set only when a Record Update action runs.
  - On the Life Sciences Cloud for Customer Engagement mobile app, fromStatus is set to the current value of the field to be updated by a Record Update action. This is set only when a Record Update action runs.
- **toStatus**: The new value of the field to be updated by a Record Update action. This is set only when a Record Update action runs.
- **fieldName**: The name of the field to be updated by a Record Update action. This is set only when a Record Update action runs.
- **actionName**: The name of the action on the action button.

Parameter	Type	Description
name	String	The name of the option to retrieve.

setOption(name, value)

Sets an environment option based on the name and value.

Parameter	Type	Description
name	String	The name of the option to set.
value	Object	The value of the option to set.

log(msg)

Logs a message to the browser console with the prefix "JS LOG: ".

Parameter	Type	Description
msg	String	The message to be logged to the browser console.

formatCustomLabel(key, defaultValue, argumentsList)

Translates a given custom label to match the current user's locale and replaces %@ symbols with given arguments. The resulting JavaScript object should be set to the title attribute in the returned message.

Parameter	Type	Description
key	String	The name of the custom label.
defaultValue	String	A default value to use when no custom label is found.
argumentsList	Array	An array of strings to use to replace placeholders in custom labels.

Database Variable

The db object provides access to the database to query records. All db functions are executed synchronously on the Life Sciences Cloud mobile app and asynchronously on desktop. In order to query correctly, use await syntax.

REQUIRED EDITIONS

Available in: Lightning Experience
Available in: **Enterprise** and **Unlimited** Editions with Life Sciences Cloud, Life Sciences Cloud for Customer Engagement Add-on license, and the Life Sciences Customer Engagement managed package.

rowById(entity, id, selectedFields)

Retrieves a record for a given object from the database by using the record ID.

Parameter	Type	Description
entity	String	The name of the object type to retrieve.
id	String	The ID of the record to retrieve.
selectedFields	Array	The array of fields to select. This parameter is supported only on desktop.

rowsByEntity(entity, selectedFields)

Retrieves records from the database for a given object.

Parameter	Type	Description
entity	String	The name of the object type to retrieve.
selectedFields	Array	The array of fields to select. This parameter is supported only on desktop.

query(entity, whereClause, selectedFields)

Retrieves records from the database for a given object based on a WHERE clause.

Parameter	Type	Description
entity	String	The name of the object type to retrieve.
whereClause	String	The SOQL WHERE clause to use in the query. You can use ConditionBuilder to build the WHERE clause.
selectedFields	Array	The array of fields to select. This parameter is supported only on desktop.

bulkQuery(queries)

Returns a key-value container `[String: [JsDbObject]]` where the key is uniqueRequestKey and the value is an array of queried records.

Parameter	Type	Description
uniqueRequestKey	String	The unique key to use to identify the specific query result.
sObjectName	String	The name of the object type to retrieve.
condition	ConditionEnhancedBuilder	The ConditionEnhancedBuilder object to use for the query.
selectedFields	Array	The array of fields to select. This parameter is supported only on desktop.

Examples

This is a `query()` example.

```
await db.query(
    "Inquiry",
    await new ConditionBuilder(
        "Inquiry",
        new FieldCondition("Type", "=", "MedicalInquiry")
    ).build(),
    ["Id", "Name", "Status", "Type"]
)

/**
 * Returns arrays of Inquiry records of from the local database. The full quer
y is:
 * "SELECT Id, Name, Status, Type FROM Inquiry WHERE Type = \'MedicalInquir
y\'"
 */
```

This is a `bulkQuery()` example.

```
let result = await db.bulkQuery([
```